

# COMP1009

# Programming Paradigms

Lecture 00-9

Interfaces, Buttons and  
Event Handlers

# Last lecture

- Layout managers
- Panels
- Label subclasses
  - Example of subclassing
- ColorLabel
  - Example of changing the 'drawmyself' method

# This Lecture

- Design Patterns
  - Layout managers are an example of the strategy pattern
- Abstract classes and Interfaces
- JButtons and event handlers
  - An example of the Observer pattern

# Design Patterns

# Patterns

- Patterns help us to identify ways in which a problem can be solved
  - We can identify the problem (pattern)
  - Then look at how people have solved this in the past
  - Much less ‘thinking’ and ‘working out’
  - Avoid potential pitfalls later on
    - Good solutions are designed to enable modification later
- There are usually many good ways to solve a problem
  - Some are usually better than others
  - Most ‘small’ problems/sub-problems that we try to solve have been solved before
- We have already seen an example of the strategy pattern but didn’t know it at the time
  - Move the decision of how to do something into another class

# Layout managers

- To lay out the components, the containers ask their layout manager to do the work
  - Important: Container does not care how layout manager does it!!!
- Container calls functions through the base class object reference to a LayoutManager (like treating Lion as Animal)
  - Different LayoutManager sub-classes re-implement these functions in different ways (to change the behaviour)
  - E.g. to lay out components one at a time in a row (flow), or to lay them out in a grid (grid) or around borders (border)
- Positioning of components:
  - Container asks layout manager to do the work
  - Layout manager asks each component for its preferences
  - Layout manager tells each component where it will be positioned
- Key design pattern feature: the container has delegated some of its work to another class – the layout manager
  - Container does not care **HOW** the layout manager does it

# Strategy Pattern - diagram

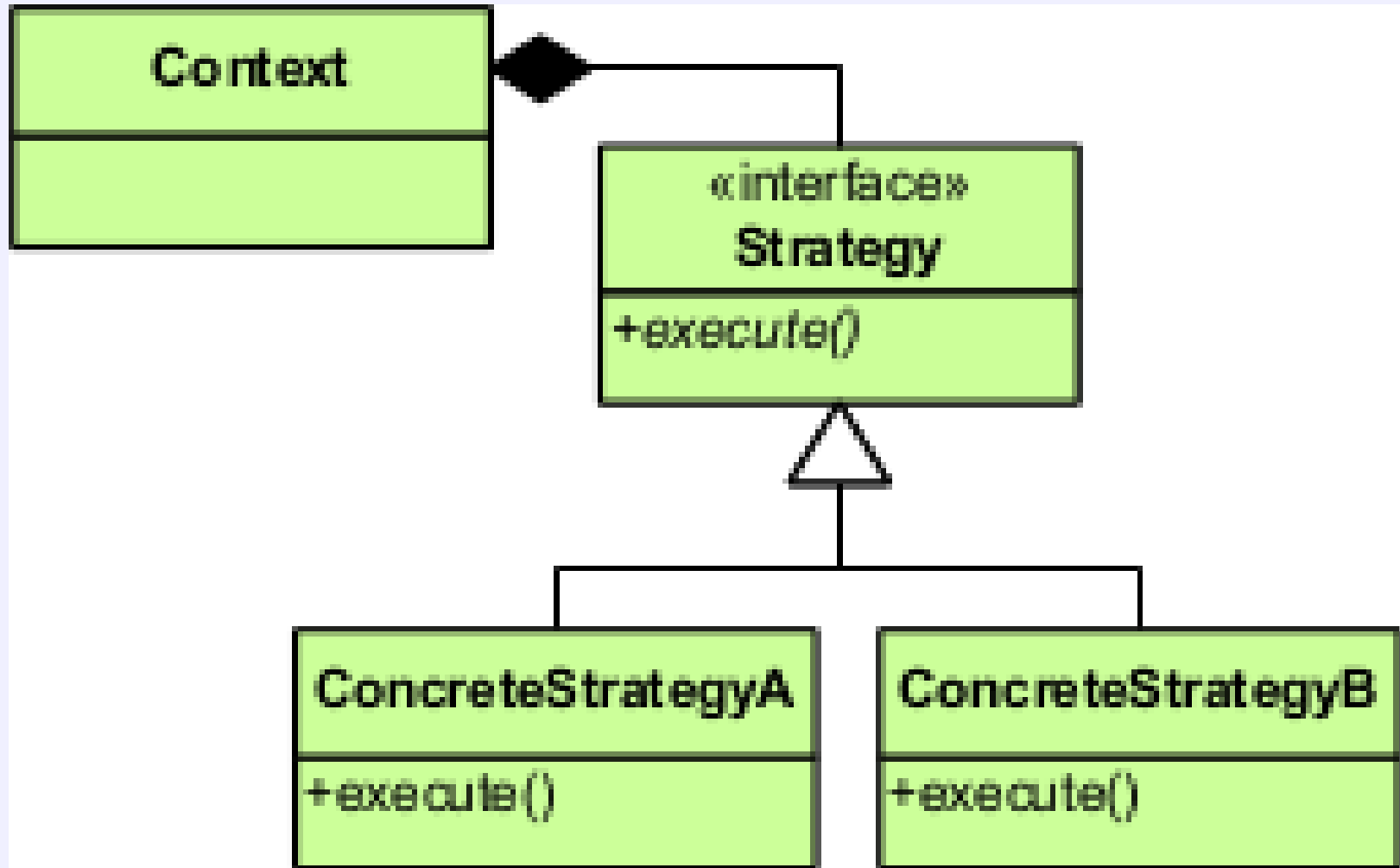
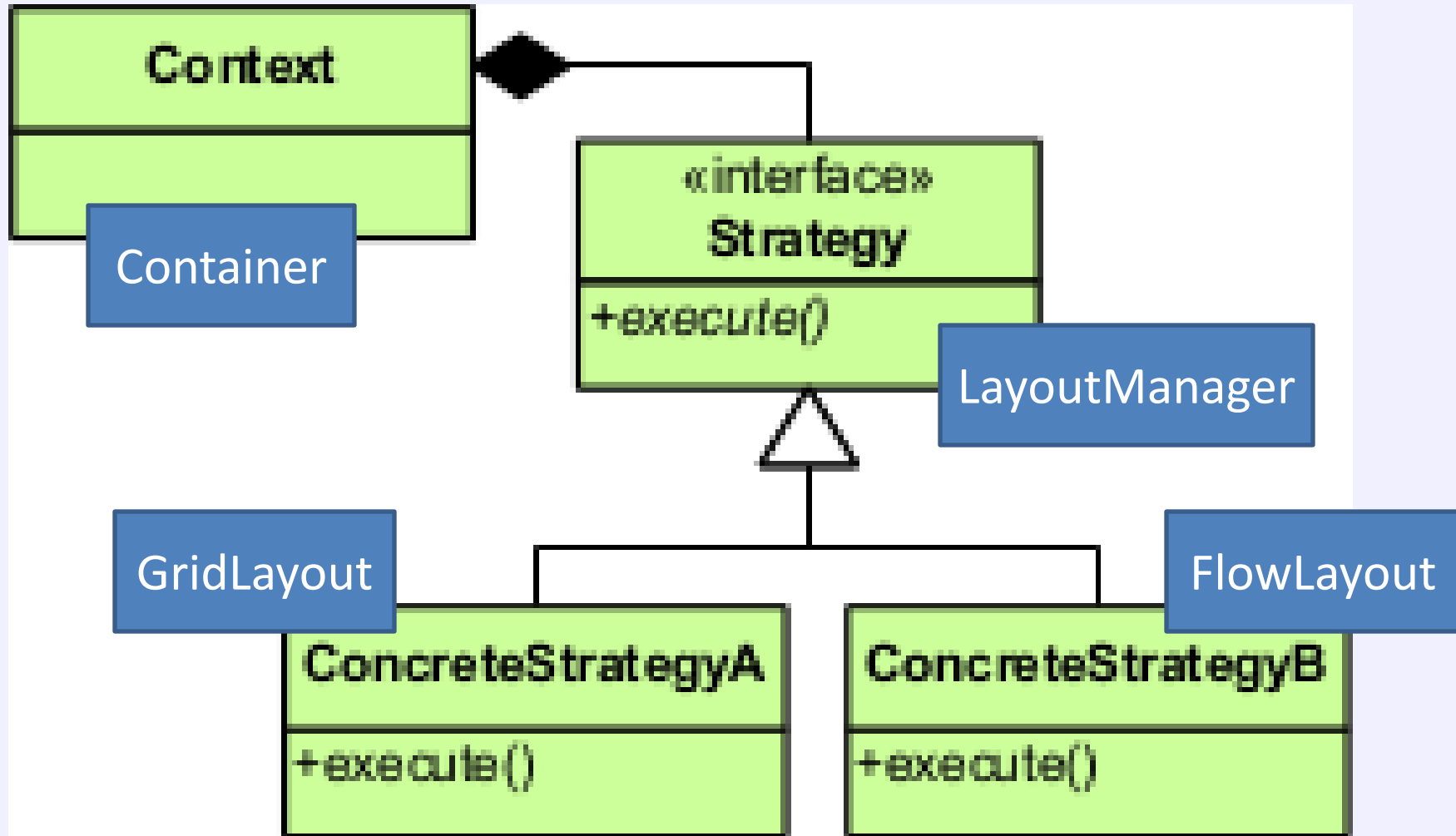


Image from wikipedia

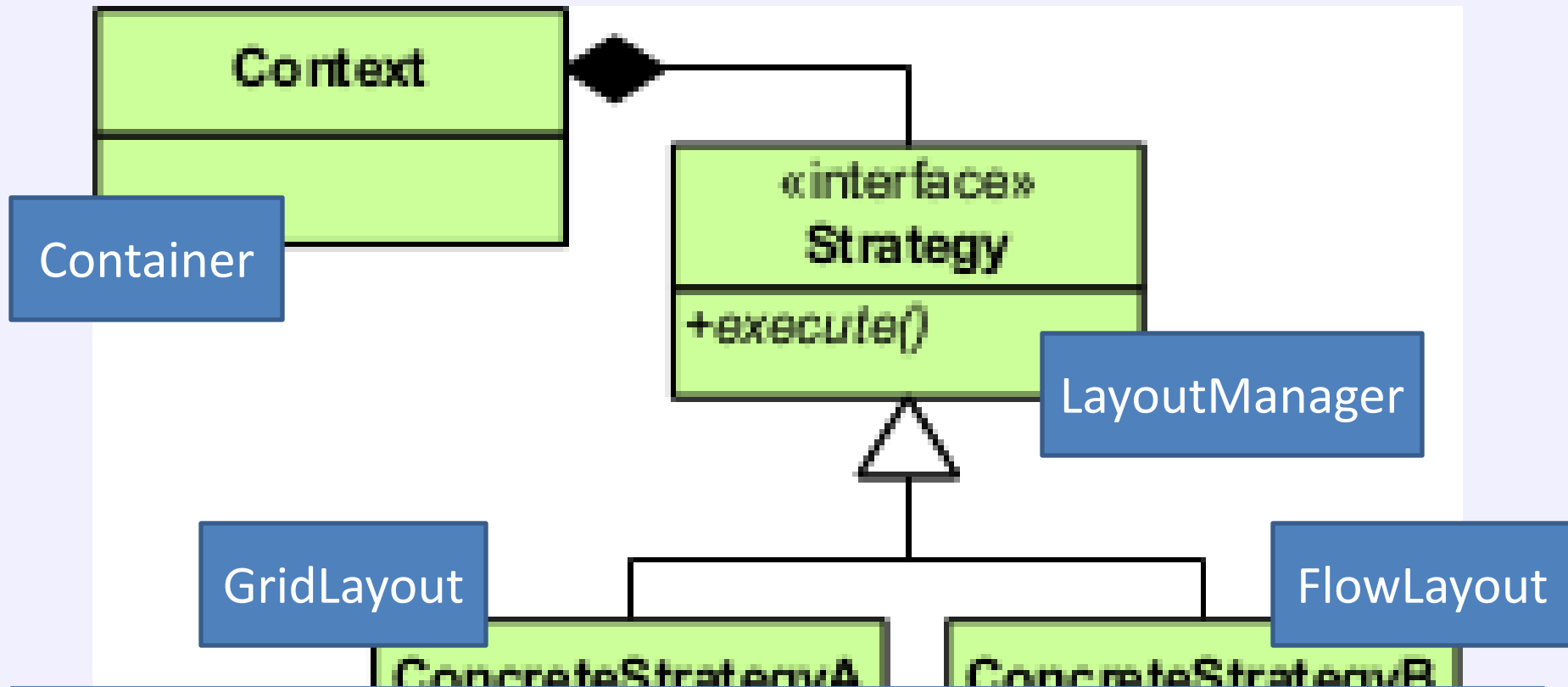
# Strategy Pattern – our classes



- We can match GUI classes against this

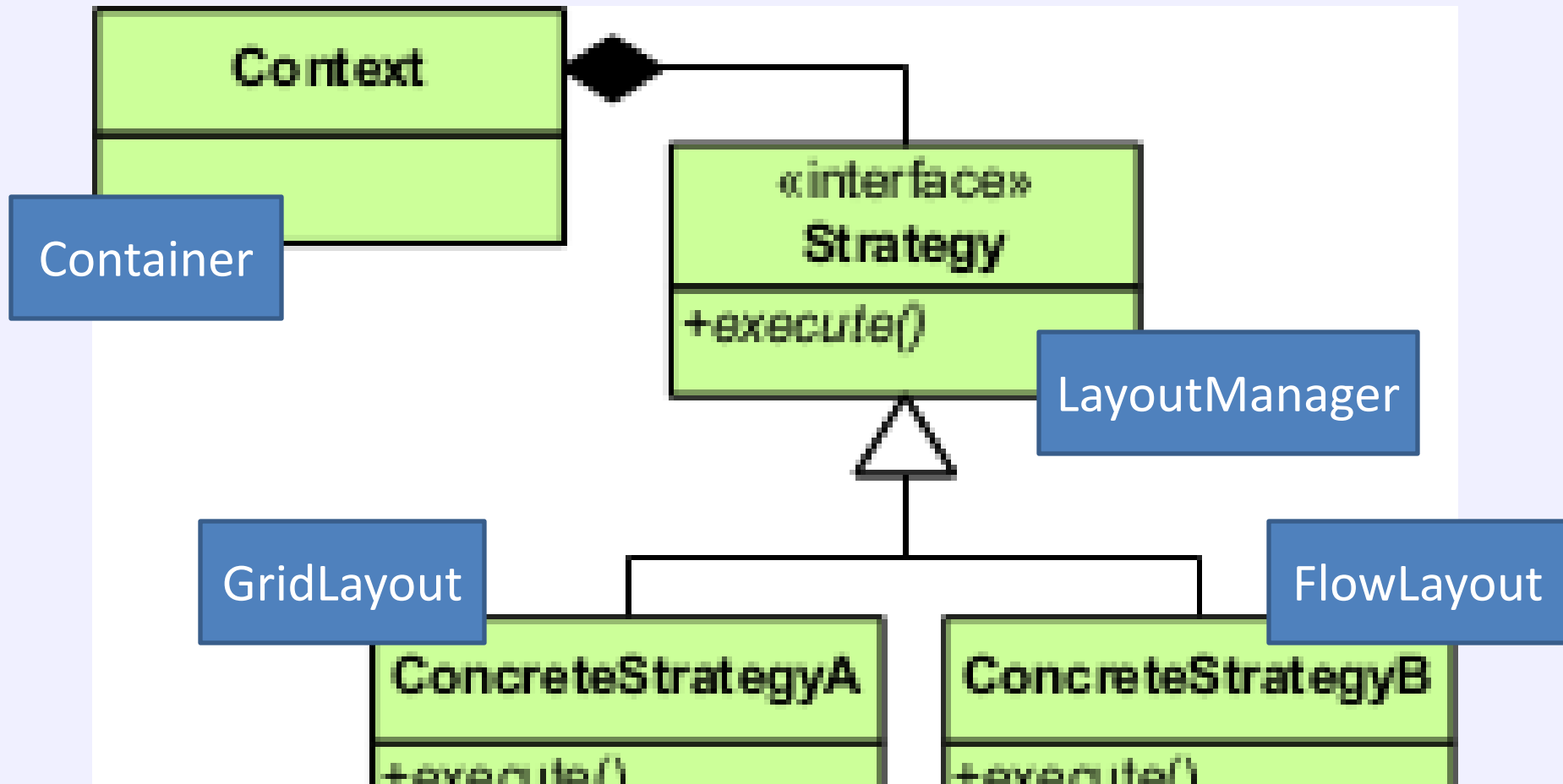


# Strategy Pattern – how it works



- Something creates a layout manager
- It passes the reference to the Container
- Container uses that reference

# Strategy Pattern - code



```
guiFrame.getContentPane().setLayout(  
    new FlowLayout());
```

# Strategy Pattern

- This is a **Behavioural Pattern**
- Allows objects' *behaviour* to be changed at runtime
- The strategies **should** be interchangeable
  - i.e. the code to use it should be the same regardless of the layout manager used
  - Which layout manager you use will change that specific behaviour of the container
- Note: not actually the case for all layout managers
  - some use a *hint* object which is passed as a parameter to the `add()` method on the container
    - E.g. `BorderLayout` needs a hint about where to place the item: north, south, east, west or centre ...
    - But take the idea of interchangeability being important

# (Sub-type) Polymorphism and Interfaces

Using the animal example  
that we had before...

# Abstract Classes and Interfaces

- We usually hide the data (implementation) inside the class
- And expose some kind of interface to the outside world
  - This is just a set of functions that others can use
- When we use polymorphism to allow behaviour to be changed at runtime, we don't care how it is implemented inside the class
- We only care **what** functions/methods are available
- If we will be implementing the methods in sub-classes then why bother defining the methods in a base class as well?
- An ***abstract*** method is a method without implementation
- Let's return to our earlier Animals example...

# Abstract classes and methods

- You can implement **some** methods and not others
- Label all methods that you do not implement as 'abstract'
- **If** you have **abstract** methods then **you cannot create instances of the class**
  - Label the class as abstract too
- E.g.:

```
public abstract class Animal
{
    public String getType()
    {
        return "Animal";
    }

    public abstract String getName();

    public abstract String getNoise();
}
```

Note: Can no longer create an instance of this class  
abstract classes are not 'real'

We will implement these in sub-classes, so no point giving an implementation here

A subclass must implement them all  
Or also be abstract

# This code is unchanged!!!

```
public class TestAnimals  
{
```

```
    public static void main(String[] args)  
    {
```

```
        Animal[] animals = new Animal[6];
```

Array of abstract  
object references

```
        animals[0] = new Bear();
```

```
        animals[1] = new Mouse();
```

```
        animals[2] = new Mouse();
```

```
        animals[3] = new Fish();
```

```
        animals[4] = new Mouse();
```

```
        animals[5] = new Bear();
```

Create objects and store  
in the array of references

We cannot create Animal objects!

```
    for ( int i = 0 ; i < animals.length ; i++ )
```

```
    {
```

```
        System.out.println( "" + animals[i].getName() +  
                             " ... " + animals[i].getNoise() );
```

```
    }
```

```
}
```

```
}
```

# Summary of interfaces

If methods are needed without an implementation  
(as a placeholder for *sub-class(es)* to implement)

Methods need to be labelled **abstract**

If a class has an **abstract** method then  
the class must be labelled **abstract** too

You cannot create instances of an  
abstract class (i.e. cannot use new)



# Interfaces

- If we don't have implementations for ***any*** of the methods, we can use an interface instead of a class
- An interface is a set of functions, **without implementations**
  - The assumption is that the classes which implement the interface will provide implementations for **all** functions
  - The subclass **MUST** implement **ALL** functions (or be abstract)
- **A subclass *implements* interfaces but *extends* a class**
- **A class can *only extend one class***
- **A class can *'implement' many interfaces***
- There are problems with ***extending*** multiple classes
  - Called 'multiple inheritance' and usually something to avoid
  - Many languages (e.g. C++) handle this, but they have to handle the complexities and ***programmers need care***
  - Using interfaces removes these problems

# Interfaces example

```
public interface Animal
{
    String getName();
    String getNoise();
}
```

- **No need to say public**
- **No implementations**
- **No data**
- **No need to say abstract**

```
public class Bear implements Animal
{
    public String getName()
    {
        return "I am a bear";
    }

    public String getNoise()
    {
        return "GROWL...";
    }
}
```

Use 'implements' for interfaces, so a Bear implements the Animal interface. This is still sub-type polymorphism!  
i.e. still IS-A conceptually

# Must change 'extends' to 'implements'

```
public class Fish implements Animal
{
    public String getName()
    {
        return "I am a fish";
    }

    public String getNoise()
    {
        return "(glug)";
    }
}

public class Mouse implements Animal
{
    public String getName()
    {
        return "I am a mouse";
    }

    public String getNoise()
    {
        return "squeek...";
    }
}
```

# This code is still unchanged!!!

```
public class TestAnimals
{
```

```
    public static void main(String[] args)
    {
```

```
        Animal[] animals = new Animal[6];
```

← Array of **interface** references

```
        animals[0] = new Bear();
```

```
        animals[1] = new Mouse();
```

```
        animals[2] = new Mouse();
```

```
        animals[3] = new Fish();
```

← Create objects and store  
in the array of **interfaces**

```
        animals[4] = new Mouse();
```

```
        animals[5] = new Bear();
```

```
    for ( int i = 0 ; i < animals.length ; i++ )
```

```
    {
```

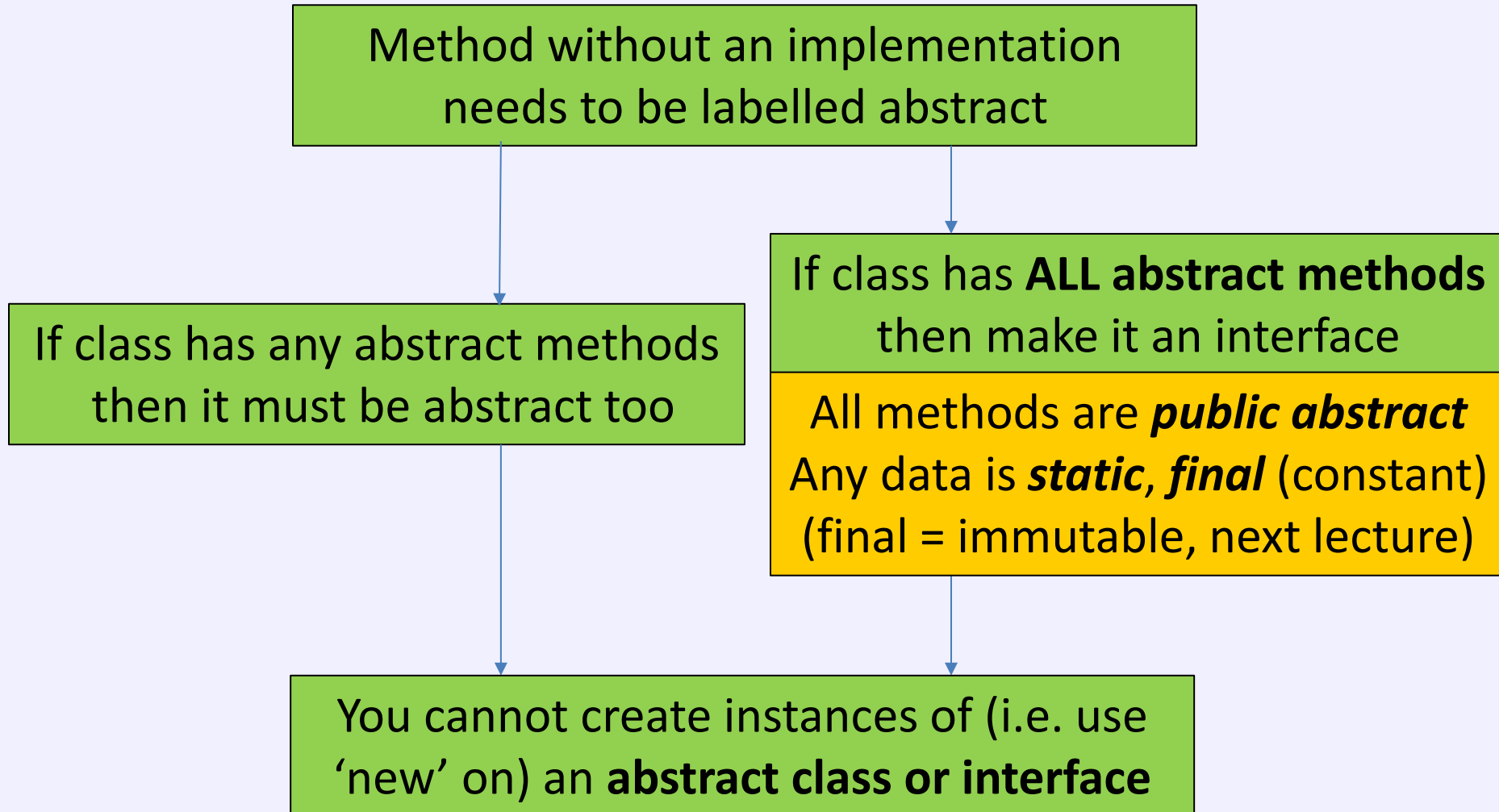
```
        System.out.println( "" + animals[i].getName() +  
                             " ... " + animals[i].getNoise() );
```

```
    }
```

```
}
```

```
}
```

# Summary



# JButtons

# JButton Example

```
public class GUIHelloWorld
{
    public static void main(String[] args)
    {
        new GUIHelloWorld();
    }

    public GUIHelloWorld()
    { // Moved content from main into here...

```

ButtonsExample1.java

- We can add buttons to our window/frame

```
JButton button1 = new JButton("Hello");  
frame.getContentPane().add(button1);
```

ButtonsExample2.java

# JButton

- We would like to allow a user to be able to do something – e.g. click something and make something else change
  - We need an ‘event handler’ to ‘handle the click’
- Java Swing uses the **Observer Pattern** for this
- This uses an Interface, the ActionListener
  - Remember: an interface is just an abstract class which we use as a superclass (base class)
  - It has all of its methods abstract
  - It’s a way to say ‘treat me as this set of functions’

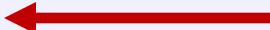


# ActionListener interface

```
package java.awt.event;
import java.util.EventListener;

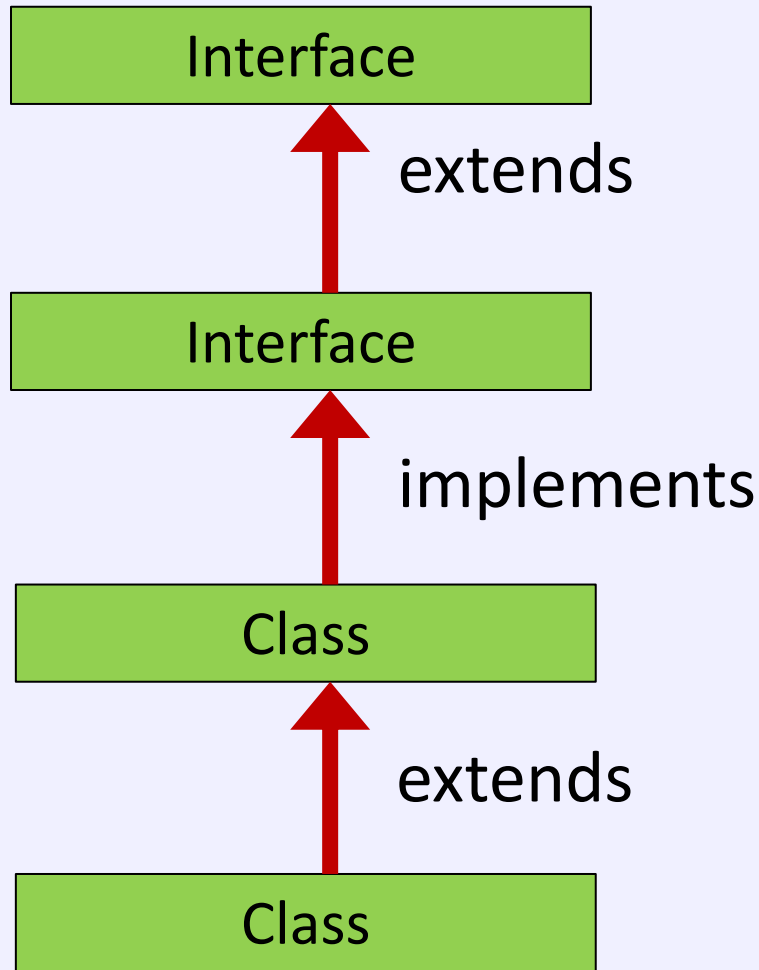
public interface ActionListener
    extends EventListener
{
    /* Invoked when an action occurs. */
    public void actionPerformed(ActionEvent e);
}
```

Note: **Interfaces**  
**extend** other  
**interfaces**, not  
implement them  
when sub-classing



- This is the real code from the default implementation
  - Reformatted, and most of the comments removed

# Classes and Interfaces



- Interface **extends** an interface
- Class **implements** an interface
- Class **extends** a class

# JButton : Main class changes

- To handle key presses, JButton needs an **object** to tell
  - We will (initially) use the object of the main class that we already have
- The class must implement ActionListener

```
public class GUIHelloWorld  
    implements ActionListener {
```

- Hint: ctrl-shift-O to add the necessary 'imports'

ButtonsExample3.java

- And implement the methods (so not abstract)
  - Hint: Quick-fix 'add unimplemented methods'

```
@Override
```

Note: @Override just tells the compiler to give us an error if this is not overriding a base class method

```
public void actionPerformed(ActionEvent e)  
{}
```

# JButton : 'listening...' (ActionListener)

- Firstly, I want to alter the button and/or label objects, so we need to have a reference to these objects somewhere that we can get at it (e.g. a member variable rather than local variable)
- Easiest thing to do is to move the object references and the initial object creation into the member variables
  - These exist for the duration of the object
  - Initialised on object creation

ButtonsExample4.java

```
public class ButtonsExample4 implements ActionListener
{
    JFrame guiFrame = new JFrame();
    JLabel label = new JLabel();
    JButton button1 = new JButton("Hello");

    public ButtonsExample4() { ...
```

# JButton : 'listening...' (ActionPerformed)

- Now tell the button that the main class wants to listen for the click events:

```
button1.addActionListener(this);
```

- And implement the actionPerformed method that you added:

```
@Override
```

```
public void actionPerformed(ActionEvent e)
{
    label.setText(label.getText()+".");
    guiFrame.pack();
}
```

ButtonsExample5.java

# Setting random colours...

- Note: for lab 4 you need to do something similar by setting a color on a ColorLabel, so it will be useful to know this
- If we want a random number, we will create a Random object, and ask it for random numbers...

```
Random rand = new Random();
```

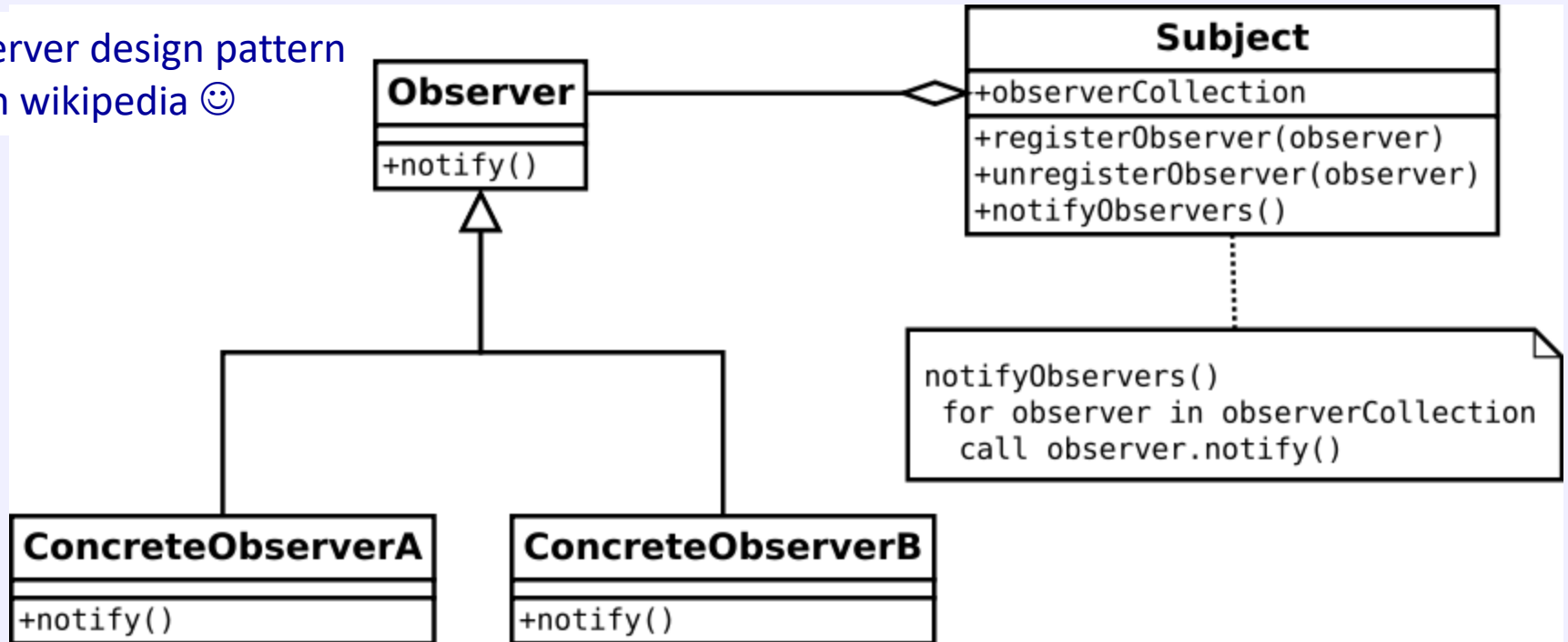
- And then use the object in your event handler...

```
button1.setBackground(  
    new Color(  
        rand.nextInt(256),  
        rand.nextInt(256),  
        rand.nextInt(256) ));
```

ButtonsExample6.java

# Observer pattern

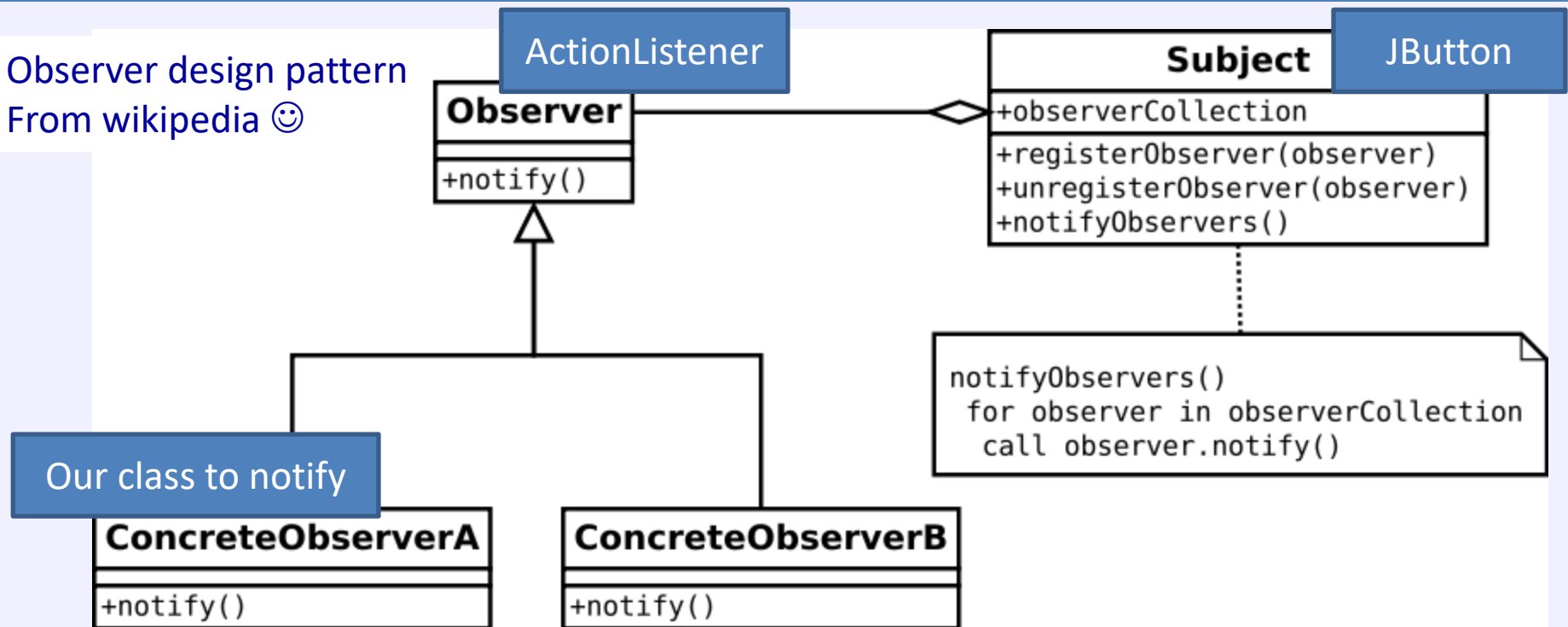
Observer design pattern  
From wikipedia ☺



- The Observer pattern is a common pattern to use when we want to notify other objects that something has happened
  - Object to notify supports some interface (Observer here)
  - Notifying object keeps a list/array/etc of there to notify
  - Notifying object considers these as the base class/interface and calls a method

# Observer pattern – our classes

Observer design pattern  
From wikipedia ☺



- The Observer pattern is a common pattern to use when we want to notify other objects that something has happened
  - Object to notify supports some interface (Observer here)
  - Notifying object keeps a list/array/etc of there to notify
  - Notifying object considers these as the base class/interface and calls a method



# Next Lecture(s)

- Lab4 is available – with an expanded ColorLabel
- Tuesday: CW2c and CW2d release
  - Lecture explaining MVC: model-view-controller pattern
  - And the 2 halves of the same Reversi program
  - Mostly live use/coding not slides

Then Friday:

- More 'button' options
  - Not needed for the lab exercise 4
- Final (immutable attributes)
- Inner Classes
- Design Patterns